

Article

Offloading Strategy Based on Graph Neural Reinforcement Learning in Mobile Edge Computing

Tao Wang [†], Xue Ouyang, Dingmi Sun, Yimin Chen and Hao Li ^{*,†}

School of Information Science and Engineering, Yunnan University, Kunming 650091, China; wangtao_6gn6@stu.ynu.edu.cn (T.W.)

* Correspondence: lihao707@ynu.edu.cn

[†] These authors contributed equally to this work.

Abstract: In the mobile edge computing (MEC) architecture, base stations with computational capabilities are subject to service coverage limitations, and the mobility of devices leads to dynamic changes in their connections, directly impacting the offloading decisions of agents. The connections between base stations and mobile devices, as well as the connections between base stations themselves, are abstracted into an MEC structural diagram due to the difficulty of deep reinforcement learning (DRL) in capturing the complex relationships between nodes and their multi-order neighboring nodes in the graph; decisions solely generated by DRL have limitations. To address this issue, this study proposes a hierarchical mechanism strategy based on Graph Neural Reinforcement Learning (M-GNRL) under multiple constraints. Specifically, the MEC structural graph constructed with the current device as an observation point aggregates to learn node features, thus comprehensively considering the contextual information of nodes, and the learned graph information serves as the environment for deep reinforcement learning, effectively integrating a graph neural network (GNN) with DRL. In the M-GNRL strategy, edge features from GNN are introduced into the architecture of the DRL network to enhance the accuracy of agents' decision-making. Additionally, this study proposes an updated algorithm to obtain graph data that change with observation points. Comparative experiments demonstrate that the M-GNRL algorithm outperforms other baseline algorithms in terms of system cost and convergence performance.



Citation: Wang, T.; Ouyang, X.; Sun, D.; Chen, Y.; Li, H. Offloading Strategy Based on Graph Neural Reinforcement Learning in Mobile Edge Computing. *Electronics* **2024**, *13*, 2387. <https://doi.org/10.3390/electronics13122387>

Academic Editor: Carlo Mastroianni

Received: 26 April 2024

Revised: 12 June 2024

Accepted: 13 June 2024

Published: 18 June 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: MEC; service coverage; deep reinforcement learning; hierarchical mechanism; graph neural network; system cost

1. Introduction

The emergence of edge intelligence, an emerging computing paradigm that pushes computing and storage resources to the network edge close to the data source, has propelled various Internet of Things (IoT) applications. In contexts such as autonomous driving [1], where vehicles need to execute intensive tasks, offloading tasks to nearby edge servers with computational capabilities [2], the efficiency of task execution is greatly enhanced. Mobile edge computing plays a key role in reducing the time and energy consumption of IoT devices in handling tasks [3], which enables rapid response to user requests and reduces network latency [4]. Applications such as Virtual Reality (VR) and Augmented Reality (AR), which are highly sensitive to latency, significantly enhance their performance by leveraging mobile edge computing [5].

In the field of mobile edge computing, two offloading modes are commonly utilized: partial offloading and 0 or 1 offloading [6,7]. The former involves dividing tasks and executing them partially on both servers and devices, while the latter executes tasks entirely locally or sends them to edge servers for execution, with no dependencies between tasks [8–12]. Most task offloading is primarily supported by deep reinforcement learning (DRL) algorithms based on policy gradients [13], aimed at maximizing long-term

rewards to avoid decisions getting trapped in local optima. However, DRL algorithms are only applicable to handling Euclidean space data, whereas irregular graph structures belong to non-Euclidean space data, which are mainly processed using graph neural networks (GNNs). Current research on computation offloading primarily addresses static scenarios, whereas, in reality, the connections between entities in offloading scenarios are dynamic; these connections are not comprehensively considered in reinforcement learning. To overcome this limitation, we propose integrating GNNs with DRL to address the offloading problem. GNNs can process entity relationship graphs in various environments, and their message-passing mechanism enables each node in the graph to incorporate the features of its neighboring nodes, thereby enhancing the model's representational capacity. The graph structures learned through GNNs can effectively serve as environments for reinforcement learning.

As depicted in Figure 1, mobile devices and base stations as network entities form a mobile edge computing (MEC) structural graph, where vertices represent base stations and mobile devices. The current device serves as the observation point within this graph; when the observation point exceeds the service range of a base station, it leads to the evolution of the graph's topological relationships. GNNs are capable of perceiving information from multi-order neighboring nodes through message passing, making them more suitable for processing dynamic multi-order graph-structured data than DRL [14]. This effectiveness is exemplified in applications such as wireless sensor networks [15] and social graphs [16].

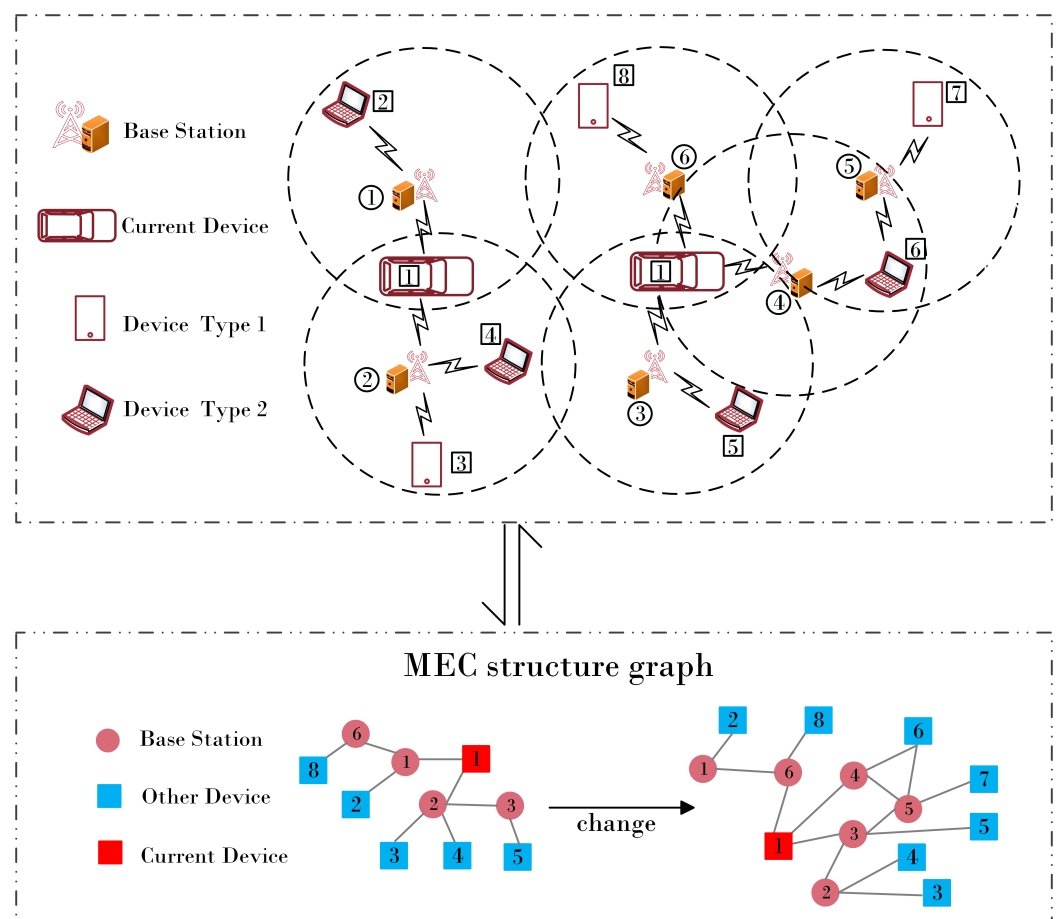


Figure 1. The mobile edge computing (MEC) scenario diagram under service coverage constraint.

Reinforcement learning is based on Markov decision processes, where the future state of agents is influenced only by the current state and the action [17]. However, in graph structures with a large number of nodes and edges, there exist complex interconnections between nodes that mutually affect each other. Consequently, these global interaction effects,

which are often ignored in traditional deep reinforcement learning methods, ultimately impact the decision-making of agents. To tackle this issue, this paper first undertakes an analysis and modeling of static topology graphs and subsequently devises update mechanisms capable of addressing dynamic changes within the graph. To facilitate the learning process aimed at understanding the connections between entities in mobile edge computing, this paper proposes an offloading strategy for Graph Neural Reinforcement Learning (M-GNRL). Considering the powerful inference ability of GNNs on graph data amalgamates DRL with GNNs, by integrating these two approaches [18], M-GNRL equips agents with the ability to effectively adapt to real-time changes in graph-structured data.

The learned node features from the MEC structural graph are utilized as the decision-making environment for the agent, the optimization objective being to minimize system cost while satisfying the service coverage constraint, where cost is defined as the weighted sum of time required for task execution and energy consumption, constituting a combinatorial optimization problem. In this study, the offloading actions are discrete; thus, Deep Q-Networks (DQNs) can be employed for resolution. To mitigate aggregation complexity, we introduce the GraphSAGE model for the sampling aggregation of the MEC structural graph. Our contributions are as follows:

- In this paper, the adjacency list method is adopted to update the graph structure. This updating scheme ensures that the proposed policy does not lose topological information when dealing with dynamic MEC graph structures. Consequently, the agents can make optimal decisions in real-time under coverage constraints, thereby achieving the minimization of system cost.
- M-GNRL employs a sampling aggregation approach for node updates to ensure weight parameter sharing among nodes within the same layer in GNN. We refrain from using attention mechanisms during the training process. This enables nodes to retain the majority of their original feature information during message propagation, thereby reducing the scale of the training parameters.
- As the learning environment of DQN is a graph, we integrate edge features from graph neural networks into the deep neural network (DNN) of DRL. Consequently, the actions generated by DQN are mapped from edge features, thereby enhancing the accuracy of actions.
- The algorithm proposed in this paper is experimentally evaluated in various scenarios, and the results indicate that M-GNRL exhibits a strong generalization ability, even in new network environments, resulting in a reduction in system costs compared to other baseline algorithms.

2. Related Works

In the real-world scenario, achieving a comprehensive minimization of latency and energy consumption may not be entirely feasible; thus, it becomes essential to consider these factors holistically and assign appropriate weights to reach an optimal system cost. Li et al. [19] proposed a distributed task offloading strategy for low-load base stations in the mobile edge computing environment, transforming the energy consumption optimization objective function into a game equation. This strategy is aimed at achieving effective savings in transmission energy by selecting base station offloading based on the game results. Gao et al. [20] extended this concept by considering the load of each edge server. They integrated the particle swarm optimization algorithm with deep reinforcement learning (DRL) to identify the optimal node for offloading, with the primary goal of minimizing latency. Lu et al. [21] introduced LSTM to improve the proposed DRL-based task offloading strategy, addressing the issue of offloading multiple service nodes and their task dependencies. Li [22] proposed a joint optimization scheme based on Deep Q-learning (DQN) to optimize offloading decisions and computing resource allocation, reducing cost. Drones also play a significant role in MEC networks, providing offloading services as edge servers in areas lacking network coverage. Liu et al. [23] proposed a DRL-based cooperative offloading strategy for drones to maximize their long-term utility. DRL offloading algo-

rithms in MEC environments have been widely discussed in the literature [24–26]. While previous strategies have shown promising results in cost reduction, they are not suitable for dynamic MEC scenarios. Hence, Ref. [27] proposed leveraging off-policy reinforcement learning supported by sequence-to-sequence (S2S) neural networks to learn an adaptive model through interaction with the MEC environment [28]. A task offloading strategy based on the DQN is proposed to address the dynamic changes in task characteristics and computing capabilities of Computing Access Points (CAPs). This strategy allows users to dynamically fine-tune the offloading ratio [29]. Considering the dynamic edge load situation, a model-free deep reinforcement learning distributed algorithm is proposed to minimize the expected cost. However, using DRL to handle complex MEC environments often overlooks the connections between network entities. For example, due to the limitation of observation, the current mobile device often can only perceive information from neighboring nodes. Consequently, capturing relationships between multi-hop nodes to make globally optimal decisions poses a challenge for DRL, in response to which graph neural networks (GNNs) have emerged.

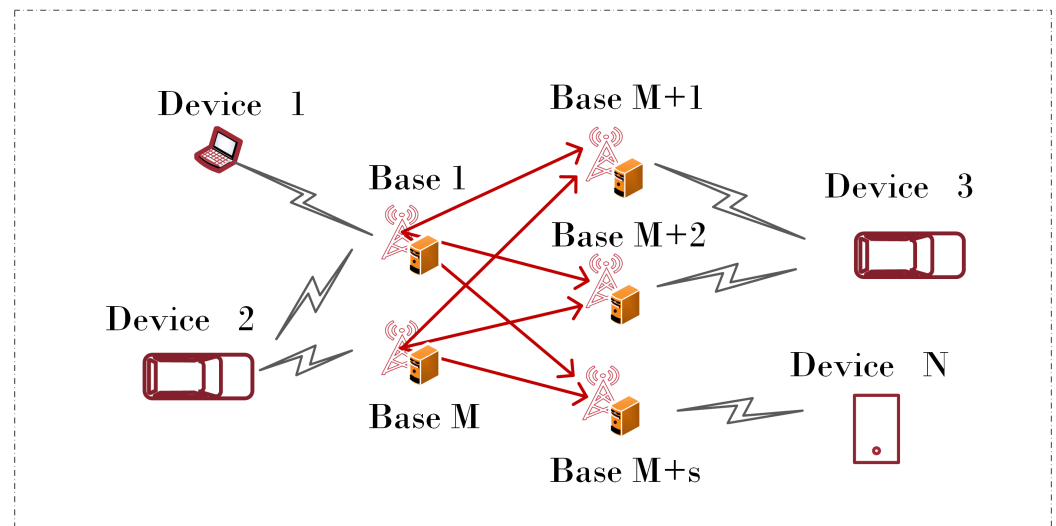
Currently, the GNN has been applied to edge computing scenarios. For example, it has been utilized in optimizing resource management in wireless IoT networks [30,31]. Since DRL often faces computational challenges when dealing with graph-structured data, many authors have introduced GNNs to jointly process the data. Li et al. [32] employed Advantage Actor-Critic (A2C) to train the speed and heading of drones, as well as their device offloading actions. Simultaneously, they utilized GNNs to supervise the training process, aiming to offload IoT device tasks to drones as much as possible. Sun et al. [33] proposed an offloading framework based on graph reinforcement learning, modeling MEC as a non-cyclic graph and conducting offloading strategies through graph state transitions, significantly reducing latency. Ref. [34] used GNN to aggregate information from the local neighborhoods of actor and critic agents, enabling the computation of agent paths in a decentralized manner using local information. The aforementioned offloading environment remains relatively static; however, in real MEC scenarios, changes can lead to irregularities in the graph nodes. Research on computational offloading often overlooks the frequent changes in the topology of the MEC structural graph; therefore, addressing how agents can make optimal decisions in dynamic environments becomes crucial. To tackle this issue, we propose a novel solution, M-GNRL, which effectively combines dynamic offloading with graph structures, greatly optimizing the offloading cost.

3. System Model

The MEC system model is shown in Figure 2. This model consists of mobile devices (MDs) and base stations (BSs), where the BSs possess computational capabilities and are capable of intercommunication. The service range of BS_m is denoted as K_m . The set of base stations at time slot t is denoted as $\mathbb{M}(t) = \{1, 2, \dots, M, M+1, \dots, M+s\}$, $m \in \mathbb{M}(t)$, where M denotes the base stations directly connected to the current mobile device, within which the current device resides within their service range, and $M+s$ denotes indirectly connected base stations, indicating that the current device exceeds their service range. We define the set of devices $\mathbb{N}(t) = \{1, 2, \dots, N\}$, $n \in \mathbb{N}(t)$, representing all mobile devices at time slot t . Each device generates only one task at the beginning of each time slot, the time slot sequence $\mathbb{T} = \{1, 2, 3, \dots, T\}$, where $t \in \mathbb{T}$, with T denoting the length of the sequence. When base stations are directly connected to a device, the agent can decide whether the task should be executed locally or offloaded. The key notations used in this article are summarized in Table 1.

Table 1. Summary of main notations.

Notation	Description
$\mathbb{N}(t)$	The set of MDs at time slot t
$\mathbb{M}(t)$	The set of BSs at time slot t
$\mathbb{T}$	The time slot sequence
B	The bandwidth
θ	The environmental impact factor
$R_{n,m}(t)$	The data transmission rate between MD_n and BS_m at time slot t
$T_n^{local}(t)$	The local execution time of task of MD_n at time slot t
$E_n^{local}(t)$	The energy consumption for local execution of task on MD_n at time slot t
$T_{n,m}^{com}(t)$	The execution time of task offloading from MD_n to BS_m at time slot t
$E_n^{tal}(t)$	The total energy consumption for task transmission for MD_n at time slot t
$Ta_n(t)$	The task generated by MD_n at time slot t
$D_n(t)$	The task size of MD_n at time slot t
$C_n(t)$	The task CPU cycle count of MD_n at time slot t
f_n^{local}	The frequency of calculation of MD_n
P_m	The transmission power of BS_m
F_m	The frequency of calculation of BS_m
$L_m(t)$	The load factor of BS_m at time slot t
Ω	The number of sample
γ	The discount factor
lr	The learning rate

**Figure 2.** The mobile edge computing (MEC) system model.

3.1. Communication Model

This section describes the upload rate during task offloading. When data transmission occurs between the device and its directly connected base station, the physical distance between them affects the transmission quality of the channel. According to the transmission rate defined in [29], the $R_{n,m}(t)$ is defined in this paper as

$$R_{n,m}(t) = B \log_2 \left(1 + \frac{P_{n,m} g_n}{N_0 + [dis_{n,m}(t)]^\theta} \right) \quad (1)$$

In this context, B represents the channel bandwidth, N_0 denotes the noise power, $P_{n,m}$ is the transmission power of device n , the channel gain is represented by g_n , and $dis_{n,m}(t)$ is the physical distance between BS_m and MD_n . θ is the environmental impact factor on the transmission rate of the device. For example, in densely built-up areas where wireless signal penetration is poor, leading to unstable links, θ will be higher. Conversely, in open areas with few obstacles and minimal interference, θ will be lower.

3.2. Computation Model

The set of tasks generated by MD_n is expressed as $Ta_n = \{Ta_n(1), Ta_n(2), \dots, Ta_n(t)\}$, and $Ta_n(t)$ indicates the task generated by MD_n at time slot t . $Ta_n(t) = [D_n(t), C_n(t)]$, where $D_n(t)$ and $C_n(t)$ represent the task size (byte) and the task CPU cycle count (cycle) generated by device n at time slot t .

Local Computing: When devices handle tasks individually, it results in associated latency, f_n^{local} represents the computing frequency of device n , and the specific execution time of tasks locally performed is represented as

$$T_n^{local}(t) = \frac{C_n(t)}{f_n^{local}} \quad (2)$$

When executing tasks locally, energy consumption is incurred, where P_n denotes the power consumption of device n , and the energy consumption is represented as

$$E_n^{local}(t) = P_n \cdot T_n^{local}(t) \quad (3)$$

Offloading Computing: When a device chooses to offload tasks to the target server directly connected to it, the resulting time is represented as

$$T_{n,m}^{trans}(t) = \frac{D_n(t)}{R_{n,m}(t)} \quad (4)$$

The energy consumption incurred during transmission is also accounted for

$$E_{n,m}^{trans}(t) = P_{n,m} \cdot T_{n,m}^{trans}(t) \quad (5)$$

It is worth noting that when tasks need to be forwarded to indirectly connected base stations for processing, $T_{M,M+1}^{trans}(t)$ represents the delay incurred in base station forwarding, P_m is the base station transmission power, and the energy consumption incurred during this period is represented as

$$E_{M,M+s}^{trans}(t) = P_m \cdot T_{M,M+s}^{trans}(t) \quad (6)$$

Therefore, the total energy consumption when tasks are ultimately offloaded to the base station is represented as

$$E_n^{tal}(t) = E_{n,m}^{trans}(t) + E_{M,M+s}^{trans}(t) \quad (7)$$

Resource scheduling: Due to the dynamic nature of devices and the random generation of tasks, base stations allocate resources to newly incoming tasks. Each device corresponds to an allocation weight $\partial_{n,m}(t)$, where F_m and $L_m^s(t)$ respectively represent the total execution frequency of BS_m and the remaining resources at time slot t , and $f_{n,m}^{com}(t)$ represents the computing frequency obtained by device n

$$f_{n,m}^{com}(t) = \partial_{n,m}(t) \cdot L_m^s(t) \quad (8)$$

$$L_m^s(t) = F_m - \sum_{q=1}^{M_n} f_{q,m}^{com}(t), q \in \mathbb{N} \quad (9)$$

Here, M_n represents the set of devices that have already obtained computational resources from BS_m .

The allocation weight is composed of the following factors:

$$\partial_{n,m}(t) = (pr_n(t), C_n(t), De_n(t)) \quad (10)$$

where $Pr_n(t)$ represents the task priority, and different types of tasks have different resource requirements. For example, tasks such as steering for vehicles or voice announcements typically have the highest priority. $De_n(t)$ indicates the deadline, specifying that tasks must be completed within a set timeframe; otherwise, they will be discarded.

The execution time of tasks can be represented as

$$T_{n,m}^{com}(t) = \frac{C_n(t)}{f_{n,m}^{com}(t)} \quad (11)$$

When performing offloading, one important metric to consider is the load factor, which reflects the resource utilization of base stations at time slot t . It is represented as follows:

$$L_m(t) = \frac{\sum_{q=1}^{M_n} f_{q,m}^{com}(t)}{F_m} \times 100\% \quad (12)$$

3.3. Problem Formulation under Multiple Constraints

When mobile devices generate tasks, choosing different execution methods will result in different system costs. In this scenario, we design an offloading strategy, which the agent uses to make decisions. According to the objective formula of [18], the system cost function for the current MD_n at time slot t is denoted as

$$\text{Cost}_n(t) = [1 - \sum_{m=1}^M x_m(t)] E_n^{local}(t) + \sum_{m=1}^M x_m(t) [\alpha E_n^{tal}(t) + \beta T_{n,m}^{com}(t)] \quad (13)$$

where $\sum_{m=1}^M x_m(t)$ with a value of 1 indicates that the task generated by the device at time slot t can only be offloaded to one base station, while a value of 0 indicates that the task is executed locally. α and β represent the weight coefficients of the transmission energy consumption and latency, respectively, satisfying $\alpha + \beta = 1$.

Under the condition of limited resources, the decisions made by agents in the MEC environment can be formulated as a minimum-cost problem. Therefore, the objective function of the convex nonlinear programming problem can be expressed as

$$\begin{aligned} & \min \& \sum_{t=1}^T \text{Cost}_n(t) \\ & \text{s.t.} \\ & \& x_m(t) \in \{0, 1\}, \quad \forall m \in \mathbb{M}(t), t \in \mathbb{T} \\ & \& \sum_{t=1}^T Ta_n(t) = T, n \in \mathbb{N}(t) \\ & \& \sum_{q=1}^{M_n} f_{q,m}^{com}(t) \leq F_m \\ & \& 0 < (1 - x_m) T_n^{local}(t) + x_m T_{n,m}^{com}(t) \leq 4\tau \\ & \& \sum_{m=1}^M x_m \in \{0, 1\} \end{aligned} \quad (14)$$

Each $x_m(t)$ is defined as 0 if the task is not offloaded to the base station m , and 1 if it is executed at BS_m . Tasks of the device can only be generated at the beginning of each time slot, and the number of tasks is denoted by T . The total computation frequency obtained by the device must not exceed the total frequency F_m of base station m . Regardless of the execution method, the maximum execution time for any task does not exceed 4τ , and each task can be offloaded to, at most, one base station.

4. Mobile Offloading Strategy Based on GNN and DRL

The offloading architecture proposed in this paper is depicted in Figure 3, consisting of the mobile device layer and the MEC layer. The mobile device layer comprises various user devices. When a task is generated at a particular time slot, the relationship between the device and the base station is transmitted to the pre-trained offloading scheduler in the form of graph-structured data for decision-making. Base stations with computational capabilities allocate resources based on task attributes, where each resource unit corresponds to a task. Finally, the execution results are returned to the device.

In a certain time slot, constructing an MEC structural graph with the current device as the observation point represents the relationship between devices and base stations; this forms a static topological graph. In this section, we investigate a strategy that combines graph neural networks with deep reinforcement learning. Graph neural networks are employed to perceive the contextual information of nodes; subsequently, the updated graph with node features serves as the environment to train the agent's actions. This strategy adopts a hierarchical mechanism.

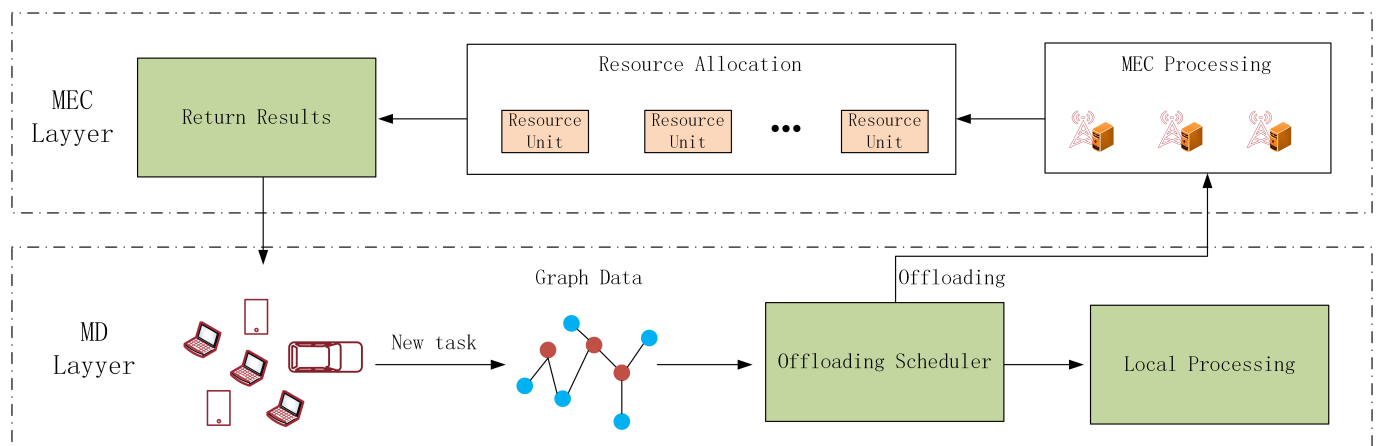


Figure 3. The proposed task offloading scheme.

4.1. M-GNRL Method

Figure 4 illustrates the training architecture of the M-GNRL strategy, where the current device is represented as $v_{n(1)}$. The graph data at time slot t are represented by an undirected graph $G_t = (V_{g(t)}, \zeta_t)$, where $V_{g(t)}$ is the vertex space, $v_{z \in \{N(t), M(t)\}}(t) \in V_{g(t)}$, the edge sets are denoted ζ_t , and $e_{n,m}^t \in \zeta_t$. The mobile device node is represented as $v_n(t)$, while the base station node is $v_m(t)$. The graph data are inputted into a GNN, which employs a multi-layer perceptron to output nodes associated with the observation node. In the agent, a deep neural network (DNN) executes actions after receiving a state vector, and the actions within the network are determined by edge features. To optimize the cumulative rewards, a loss function is utilized to update the network parameters and the policy with the reward feedback from the environment serving as input to the function. This iterative process results in the identification of the optimal policy. Ultimately, the quadruple (s, a, r, s') consisting of the current state s , action a , reward r , and next state s' is stored within an experience replay pool to train the agent effectively.

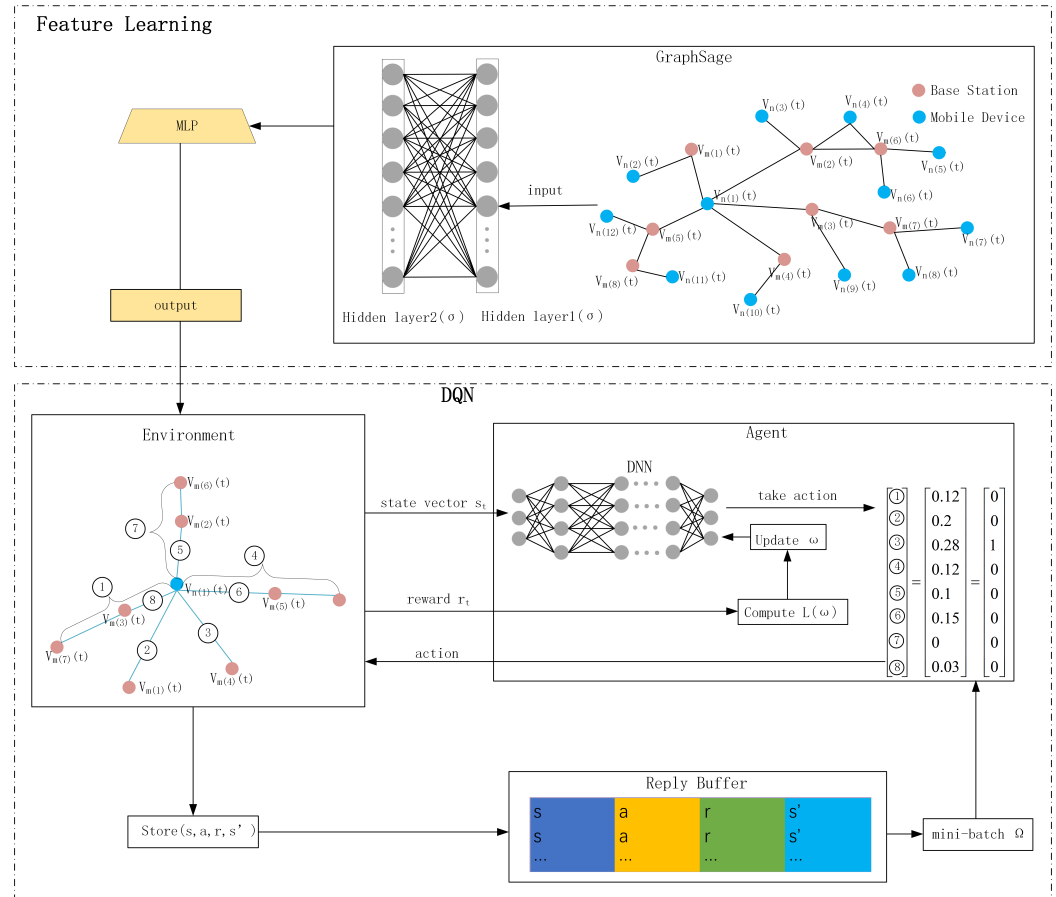


Figure 4. The M-GNRL training framework.

4.2. Learning Node Features

To process the raw graph structural data, updating node features is necessary. In this paper, aggregation updates are performed through sampling. Here, $v_{z(i)}(t)$ is used to represent the nodes to be learned in the i -th layer. The updated features are then utilized as inputs for subsequent layers. In the graph neural network, there exists a total of I hidden layers, with the 0-th layer serving as the input layer. Since this paper only involves offloading to directly or indirectly connected base stations, the current observation node only needs to know information about nodes within two hops. Therefore, $I = 2$, derived from the equation in [33], and the embedding of device node n at the i -th layer is obtained as

$$h_n^i(t) = \sigma(W_1^i \cdot h_n^{i-1} + \sum_{m \in N_n} \frac{1}{\sqrt{|N_n|}\sqrt{|N_m|}} [W_1^i \cdot h_m^{i-1} + W_2^i \cdot (h_n^{i-1} \odot h_m^{i-1})]) \quad (15)$$

where W_1^i and W_2^i represent the trainable weight parameters. Let $h_n^i(h_m^i)$ represent the embedding of a node, and R^d denote the embedding dimensionality. N_n represents the set of sampled base station nodes for node n , and N_m is the set of sampled device nodes for node m . $|N_n|$ and $|N_m|$ are the numbers of sampled neighboring nodes, and σ is the activation function.

In order to accurately understand the features of base station nodes, it is crucial to gather relevant information from various viewpoints, including adjacent base stations and devices. The embedding for base station node m is provided below [33]

$$h_m^i(t) = \sigma(W \cdot \text{Con}(h_{B(m)}^i(t), h_{U(m)}^i(t))) \quad (16)$$

$$h_{U(m)}^i(t) = \sigma(W_1^i \cdot h_m^{i-1} + \sum_{n \in N_m} \frac{1}{\sqrt{|N_m|}} (W_2^i \cdot h_n^{i-1})) \quad (17)$$

$$h_{B(m)}^i(t) = \sigma(W_1^i \cdot h_m^{i-1} + \sum_{q \in Z_m} \frac{1}{\sqrt{|Z_m|}} (W_2^i \cdot h_q^{i-1})) \quad (18)$$

Here, the concatenation function, denoted as CON, combines two learned features, which are then mapped through an activation function to form the final base station node feature. $h_{U(m)}^i(t)$ represents the features aggregated from device nodes, $h_{B(m)}^i(t)$ represents the features aggregated from base station nodes, Z_m and $|Z_m|$ are the set of neighboring base stations and the number of samples taken, respectively.

When only devices exist around BS, we can update node features using the following approach:

$$h_m^i(t) = \sigma(W_1^i \cdot h_m^{i-1} + \sum_{n \in N_m} \frac{1}{\sqrt{|N_n|} \sqrt{|N_m|}} [W_1^i \cdot h_n^{i-1} + W_2^i \cdot (h_n^{i-1} \odot h_m^{i-1})]) \quad (19)$$

Meanwhile, during backpropagation, a loss function regarding parameters W_1^i and W_2^i is derived. The objective is to make neighboring nodes have similar embeddings as much as possible, so we optimize the parameters using the Adam algorithm. Finally, after updating the nodes through GraphSAGE, they will be inputted into a multi-layer perceptron (MLP), through forward propagation, and new nodes related to the observation node will be obtained. The loss function can be obtained from the following equation [32]:

$$\varphi(z_{u(i)}) = -\log(\sigma(z_{u(i)}^T z_{v(i)})) - Q E_{v_n \sim P_n(v)} \log(\sigma(z_{u(i)}^T z_{v_n(i)})) \quad (20)$$

where $z_{u(i)}$ represents the embedding of node u at the i -th layer, $v(i)$ is a randomly walked neighboring node in the i -th layer, $v_n \sim P_n(v)$ represents the probability distribution of negative sampling sample nodes, and Q is the number of negative sampling samples.

4.3. Offloading Strategy

In this context, the maximum execution time for each task is set as $\tau_n(t)$. Here, T^e represents the actual time taken for the task to execute either locally or when offloaded. The term $|De_n(t) - t - T_{n,m}^{trans} - T_{M,M+s}^{trans}(t)|$ represents the allowable execution time after task offloading and before the deadline, and $|De_n(t) - t|$ denotes the limit on time for local execution before the deadline. Furthermore, tasks are discarded if they are not fully processed within the maximum execution time. We set $T^e = \{T_n^{\text{local}}(t), T_{n,m}^{\text{com}}(t)\}$, $\tau_n(t) = \min\{|De_n(t) - t - T_{n,m}^{trans}(t) - T_{M,M+s}^{trans}(t)|, 4\tau\}$ or $\tau_n(t) = \min\{|De_n(t) - t|, 4\tau\}$. The reward function for the agent's actions is as follows:

$$\begin{cases} r_t = \frac{1}{\text{Cost}_n(t)} & \& 0 < T^e \leq \tau_n(t) \\ r_t = 0.1 * (\tau_n(t) - T^e) & \& \text{otherwise} \end{cases} \quad (21)$$

At time slot t , the state vector s_t of the environment is composed of task size, CPU cycles, task deadline, base station load rate, and its remaining resources, where $\mu(m)$ denotes the base station set of nodes

$$s_t = \{D_n(t), C_n(t), De_n(t), L_{\mu(m)}(t), L_{\mu(m)}^s(t)\}, n \in \mathbb{N}(t), \mu(m) \in \mathbb{M}(t) \quad (22)$$

In this study, the reinforcement learning environment is represented by a mobile edge computing (MEC) structural diagram. Consequently, we introduce the concept of edge features from graph neural networks into the reinforcement learning network, where edges represent the probability of offloading device tasks to base stations. Using a concatenation function, we combine the features of two nodes as inputs to MLP_1 (Multilayer Perceptron), and the output of MLP_1 is a feature vector, while the output of MLP_2 is a real value. Finally,

the offloading probability is obtained through the softmax function. Here, $h_n(t)$ and $h_m(t)$ represent the updated features as depicted below:

$$h_{e_{n,m}^t} = \text{Softmax}(MLP_2(\sigma(MLP_1(\text{Con}(h_n(t), h_m(t))))) \quad (23)$$

The action a_t taken at time step t is represented as a vector, which is obtained through the mapping of edge feature $\hat{X}(t) = \{x_1(t), x_2(t), \dots, x_M(t), x_{M+1}(t), \dots, x_{M+s}(t)\}$. The action space A is represented as

$$A = \{a_t | a_t = \hat{X}(t)\}, t \in \mathbb{T} \quad (24)$$

The decision vector is represented using one-hot encoding, where the agent selects the base station for offloading corresponding to the edge with the highest probability. Specifically, $x_m(t)$ is as follows

$$x_m(t) = \begin{cases} 1, \max[h_{e_{n,m}^t}] \\ 0, h_{e_{n,m}^t} < \max[h_{e_{n,m}^t}] \end{cases} \quad (25)$$

Upon receiving the decision information, the environment assigns reward values to the agent to update parameters in the calculation of the loss function. The goal is to minimize the loss value. The loss function defined in [22] can be utilized in this study as shown below, which is expressed as follows:

$$L(\omega) = \frac{1}{\Omega} \sum_{j=1}^{\Omega} [(r_j + \gamma Q(s_{j+1}, a_{j+1}, \omega) - Q(s_j, a_j, \omega))^2] \quad (26)$$

Ω denotes the number of samples extracted from the experience replay pool, ω representing the updated parameter and γ indicating the discount factor, and Q denotes the action value function.

We use the gradient descent method for parameter iteration. In the equation, lr stands for the learning rate, and the formula is expressed as follows:

$$\omega = \omega - lr \cdot \nabla_{\omega} L(\omega) \quad (27)$$

For the decision-making process, the method utilized is depicted in Algorithm 1, where the parameters are determined after training. Ultimately, the agent arrives at an optimal decision

$$a_t^* = \arg \min_{a_t \in \hat{X}_m(t)} \sum_{t=1}^T \text{Cost}_n(t) \quad (28)$$

Algorithm 1 has shown promising results in experiments. However, this study focuses on task offloading for a single agent. Therefore, to enhance the adaptability of the algorithm, future work should consider the collaboration and competition among agents. This expansion aims to enable the extended algorithm to make decisions simultaneously for multiple agents, meeting the demands of complex task scenarios.

Algorithm 1 M-GNRL for task offloading strategy.**Input:** Graph Information $V_g(t)$, ζ_t , training episodes ep **Output:** Optimal decision a_t^*

```

1: for  $i = 1$  to  $I$  do
2:   for  $v_{z(i)}(t) \in V_g(t)$  do
3:     if  $z = n$  then
4:       Get node embedding via Equation (15);
5:     else
6:       Compute node embedding via Equations (16)–(19);
7:     end if
8:   end for
9:   Compute the loss function Equation (20) and optimize the parameters  $W_1^i$  and  $W_2^i$  using the Adam algorithm;
10: end for
11: The learned graph information serves as the environment for deep reinforcement learning;
12: Initialize experience replay pool;
13: for  $episode \leftarrow 1$  to  $ep$  do
14:   for  $t = 1$  to  $T'$  do
15:     Input state vector by Equation (22);
16:     Calculate unloading probability in Equation (23)
17:     Take action in Equation (24);
18:     Store action, state, reward, next state information into experience replay pool;
19:     Retrieve samples for training, and update parameters  $\omega$  through the computation of loss function Equation (26);
20:   end for
21: end for

```

4.4. Update Graph Structure

In the context of MEC, mapping scenarios to a graph structure does not necessitate rebuilding from scratch each time; instead, we can perform localized updates based on cached graph information. As illustrated in Figure 5, at time slots t , $t + 5$, and $t + 10$, the positions of the observation node have changed, leading to alterations in the graph structure and its node attributes. We employ an adjacency list approach for updating the graph structure.

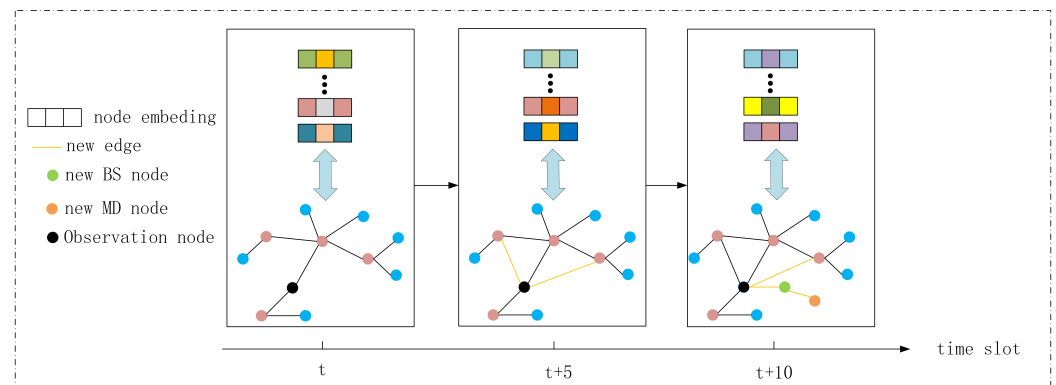


Figure 5. On the change of graph structure in different time slots.

For graph neural networks, the adjacency list method is widely used to represent the connectivity of graphs, where updating the graph structure involves manipulating linked lists without affecting the representation of the entire graph. Simultaneously, whereas each node in the graph maintains a linked list containing other nodes connected to it, the adjacency list only stores the edges that actually exist, thus saving space more effectively. When adding an edge, the new adjacent node is appended to the linked list of the source

node. Similarly, to remove an edge, the corresponding adjacent node can be deleted from the source node's linked list. Furthermore, modifying the weight of an edge can be achieved by updating the information of the respective node in the linked list, demonstrating the efficiency of updating the graph connectivity without impacting other aspects.

The algorithm presented in Algorithm 2 serves as a demonstration utilizing the addition case, while similar procedures are employed for deletion instances. The process of updating the graph structure commences with the creation of an initialized linked list $\&G(t)$, and for each vertex in the graph, a corresponding vertex table $\&Node(v_z(t))$ is established. Simultaneously, the adjacent vertex set $v_z^{adj}(t)$ is determined, which is crucial for the addition of edge tables $\&Edge(v_z(t), v(t))$. In the context of the predefined service range, if the graph structure undergoes modifications from time slot t to $t + n$, we need to obtain the set of added vertices $C(v_{t+n})$ and the set of edge information alterations $C(e_{t+n})$. Subsequent to this, a series of operations is executed to update the linked list $\&G(t + n - 1)$, leading to the creation of a new linked list $\&G(t + n)$. This updated graph structure is then utilized as input for Algorithm 1 to facilitate an optimal decision-making process.

Algorithm 2 Update graph structure.

Input: $V_g(t)$, ζ_t , set of changing vertices $C(v_{t+n})$ and their edge set $C(e_{t+n})$

Output: Linked lists $\&G(t)$, $\&G(t + n)$

```

1: for  $v_z(t) \in V_g(t)$  do
2:   Construct vertex table  $\&Node(v_z(t))$ ;
3:   for  $v(t) \in v_z^{adj}(t)$  do
4:     Add edge table  $\&Edge(v_z(t), v(t))$ ;
5:   end for
6: end for
7: Get initialized adjacency linked list  $\&G(t)$ ;
8: for  $n = 1$  to  $K$  do
9:   if  $\&G(t + n - 1) \neq \&G(t + n)$  then
10:    for  $v_z(t + n) \in C(v_{t+n})$  do
11:       $\&G(t + n - 1)$  adds vertex table  $\&Node(v_z(t + n))$ ;
12:    end for
13:   end if
14: end for
15: Obtain the new adjacency linked list  $\&G(t + n)$ ;

```

5. Experimental Analysis

5.1. Simulation Experiment Setup

The dataset utilized in this study for conducting simulation experiments consists of three main components: device data, base station data, and the connection information between them. Each record in the device data is represented as $[ID, D_n(t), C_n(t), De_n(t), Pr_n(t), f_n^{local}, timeslot\ t]$. While the base station data take the form $[ID, F_m, P_m, L_m(t), L_m^s(t), K_m, timeslot\ t]$, the connection information is denoted as $[base\ station\ ID, device\ ID\ or\ another\ base\ station\ ID, data\ transmission\ rate, physical\ distance\ between\ them, timeslot\ t]$. When experimenting with M-GNRL, it is crucial to consider scenarios involving multiple base stations and devices, with mobile devices uniformly distributed within the service range of each base station.

Table 2 outlines the data scope and its parameters for the experiments. The number of tasks is represented by the length of the time slot sequence, denoted as T , where each time slot begins with the generation of a single task. The bandwidth B , along with its environmental coefficient θ , affects the transmission rate between the base station and the device. Parameters such as K_m , $D_n(t)$, $C_n(t)$, f_n^{local} , and F_m are all part of the simulation dataset, with selected key data presented in Table 2. In the reinforcement learning model, parameters such as training episodes ep , sample size Ω , experience replay pool capacity, discount rate γ , and learning rate lr are essential. Furthermore, subsequent experiments

will investigate the impact of ep and γ on reinforcement learning rewards. In order to evaluate the performance of M-GNRL, this study necessitates comparing M-GNRL with baseline algorithms. The descriptions of these algorithms are as follows:

- *LOCAL*: The agent will only select tasks to be executed on the device.
- *RANDOM*: The decision of task offloading and base station selection are both made randomly.
- *GNN-A2C*: The author's deep-graph-based reinforcement learning framework, which employs graph neural networks to supervise the action training of unmanned aerial vehicles in Advantage Actor-Critic (A2C) methods. This framework achieves rapid convergence and significantly reduces the task missing rate in aerial edge Internet of Things (IoT) scenarios [32].
- *Coop-UEC*: Drones are capable of collaborating to offload computing tasks. In order to maximize long-term rewards, the author formulates an optimization problem, describing it as a semi-Markov process and proposing a DRL-based offloading algorithm [23].

Table 2. Data scope and parameters setting.

Parameter and Values
bandwidth B : 4 Mhz
length of time slot sequence T : 80
environment influence coefficient θ : [0.5, 1]
service range of the base station m K_m : [0.5, 4]/km
task size $D_n(t)$: [800, 2000]/kbytes
task CPU cycle $C_n(t)$: [1000, 2500]/Mcycles
computational capability of device n f_n^{local} : [0.5, 1.5]/Ghz
computational capability of base station m F_m : [4, 11]/Ghz
training episodes ep : 1000, 1200, 1400
batch size Ω : 100
number of tuples in the experience pool: 1500
discount factor γ : 0.93, 0.95, 0.97, 0.99
learning rate lr : 0.001

5.2. Simulation Scenario

In the initialized simulation environment, base stations and devices are assumed to be deployed within a 4 km $\times$ 4 km area as illustrated in Figure 6. The service coverage of each base station is set to 0.5 km. The figure shows the current device mobility trajectory and the distribution of other devices and base stations. The movement of the current device leads to ongoing changes in the interacting base stations. The agent can identify offloading points using the trained policy to optimize the reduction in system cost and enhance the task arrival rate, thereby validating the subsequent experimental outcomes.

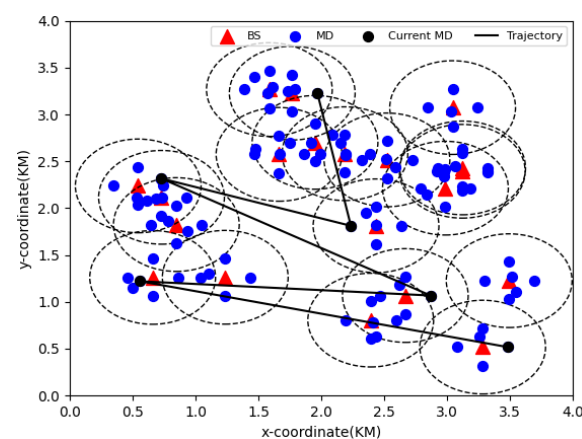


Figure 6. Distribution of mobile devices and base stations.

5.3. Performance Analysis of Different Strategies

For this study, performance metrics such as convergence speed, task arrival rate, and system cost are evaluated to compare different methods and reflect the performance of each strategy. To ensure the reliability of the results, all comparative experiments in this study are conducted using the method of controlling variables.

In reinforcement learning, the discount factor γ , within the range of 0 to 1, plays a crucial role in computing the cumulative reward value. A value of $\gamma = 0$ implies that the agent's focus is solely on immediate rewards, resulting in decisions that are locally optimal. Conversely, when $\gamma = 1$, convergence issues may emerge, underscoring the importance of selecting an appropriate discount factor for policy training. Figure 7 illustrates the impact of different γ values (0.93, 0.95, 0.97, 0.99) on the average reward. Interestingly, the increase in average reward does not exhibit a linear correlation with the number of episodes, suggesting that excessive training could lead to reduced rewards. Notably, the figure indicates a non-linear association between the discount factor and the average reward, with the highest average reward achieved when $\gamma = 0.97$, outperforming other values. Consequently, for subsequent experiments, we decide to adopt this specific γ value.

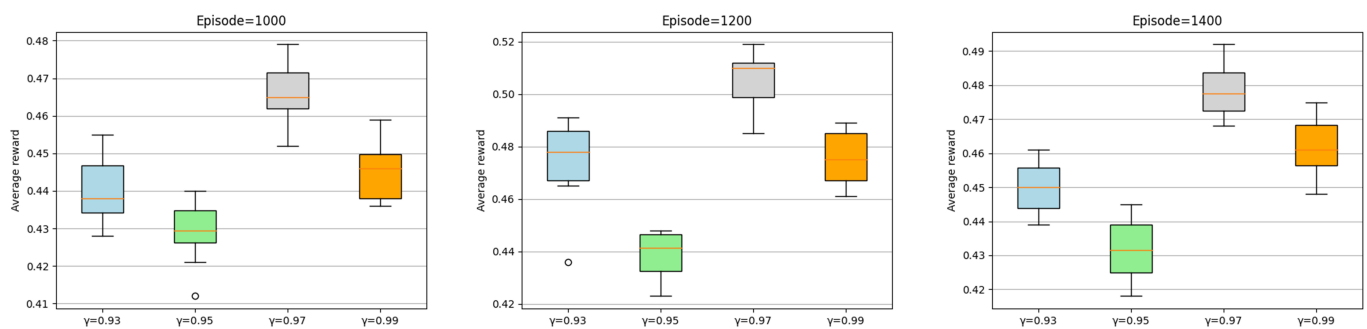


Figure 7. Impact of discount factor on average reward.

A comparative analysis of the convergence performance of Coop-UEC, GNN-A2C, and M-GNRL is illustrated in Figure 8 during their training processes. Their cumulative rewards showcase a consistent upward trend, eventually converging as the number of iterations reaches a certain threshold. From the analysis, (a) it is observed that after 1050 episodes, the cumulative reward values fluctuate within the range of 0.35 to 0.44; (b) following 920 iterations, the cumulative rewards stabilize, fluctuating between 0.66 and 0.78; and (c) upon reaching 950 iterations, the cumulative rewards fluctuate between 0.82 and 0.97. GNN-A2C demonstrates superior convergence speed and post-convergence reward values over Coop-UEC. Moreover, GNN-A2C shows a slightly faster convergence speed than M-GNRL; however, as the number of episodes increases, the disparity in convergence performance between them becomes evident. Notably, when reaching a stable state, M-GNRL achieves higher cumulative reward values than GNN-A2C.

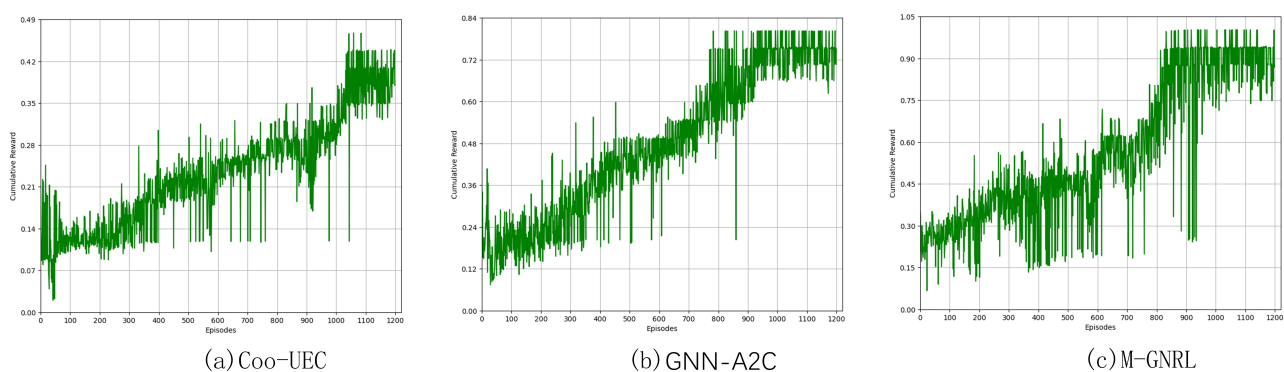


Figure 8. Convergence performance of different algorithms.

The task arrival rate is utilized to measure the scenario where all device tasks are offloaded, executed, and returned in a particular time slot. In Figure 9a, the offloading base station is determined according to the algorithm, where the task volume gradually increases while the base station resources remain fixed. This may result in a reduction in allocated computational resources, leading to task timeouts and subsequent discarding. Figure 9b explores six MEC scenarios, denoted by the topology structure $(4 + 5) * 20$, representing the numbers of directly and indirectly connected base stations and mobile devices, respectively. From the figure, it is evident that the base station load rate tends to increase with the number of devices, causing a decrease in the frequency of task acquisition per device and consequently longer task execution times, thereby impacting the task arrival rate. Initially, the first three strategies in Figure 9b show no significant differences. However, as the number of devices increases, Coop-UEC exhibits relative weaknesses in handling complex graph structures. In contrast, the aggregation nature of graph neural networks enables current mobile devices to perceive global information, allowing M-GNRL to consider offloading targets more comprehensively than Coop-UEC. Due to the incorporation of edge features into the network of reinforcement learning, M-GNRL offers more precise policy selection compared to GNN-A2C, and ultimately, M-GNRL achieves a higher arrival rate than GNN-A2C and Coop-UEC.

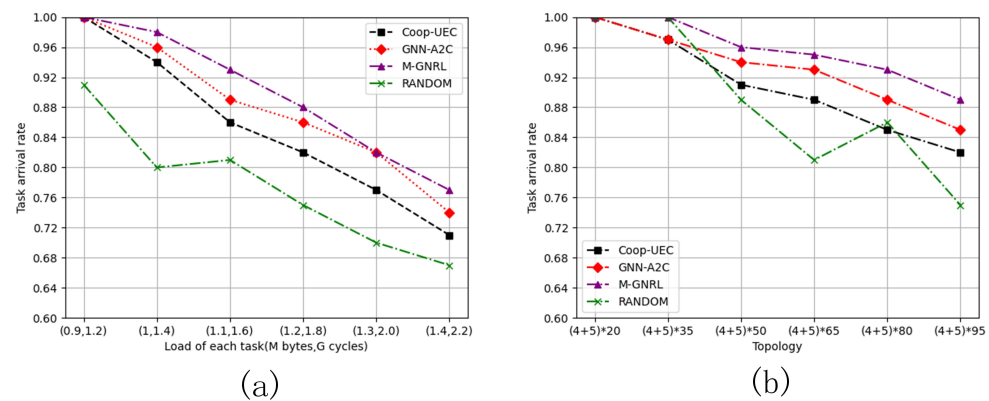


Figure 9. Task arrival rate. (a) Task arrival rate for different task load; (b) Task arrival rate in different topology.

In real scenarios, some devices share a common base station. Let us select one device as the experimental subject. As depicted in Figure 10b, the initial settings for network topology, base station service range, and base station operating frequency are set to $(4 + 5) * 95$, 0.5 km, and 5 GHz, respectively. In Figure 10a, a reduction in the overall number of devices increases the frequency at which the experimental subject acquires resources, thereby reducing system cost. Figure 10b reveals that expanding the service radius allows more base stations to directly connect with the current device. This not only reduces the transfer time and energy consumption but also enables the agent to select the optimal node for offloading from a wider range, thus achieving the desired outcome. In Figure 10c, a significant increase in the base station operating frequency significantly reduces the system cost associated with tasks. Overall, by averaging the cost differences obtained in the same experimental environment, it can be inferred that the effects of random offloading and local execution are far inferior to the other three methods. GNN-A2C reduces system cost by approximately 22.8% compared to Coop-UEC, while M-GNRL achieves a reduction of about 15.6% in system cost compared to GNN-A2C.

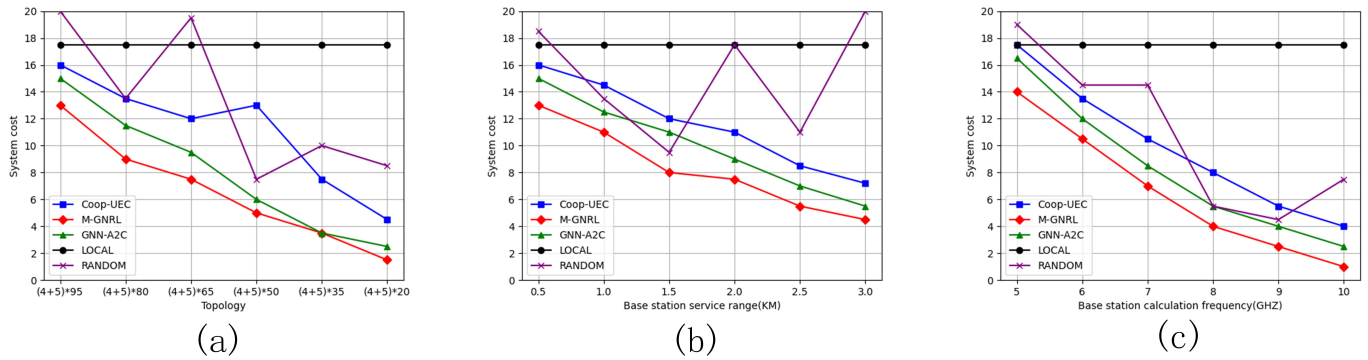


Figure 10. System cost generated by selecting different strategies. (a) System cost under the influence of topology; (b) System cost under the influence of base station coverage; (c) System cost under the influence of base station calculation frequency.

Figure 11 compares M-GNRL, selecting three sets of weight coefficients with $\alpha = 0$, $\beta = 1$, and $\alpha = 1$, $\beta = 0$. For these two scenarios, tasks 1–6 adopt the values from Figure 9a. It is observed that considering only one-sided factors for task offloading leads to extreme cost values, resulting in uneven resource allocation, where some base stations are excessively utilized while others remain inefficient. To effectively reduce the cost of offloading, it is necessary to balance the proportions of cost-influencing factors.

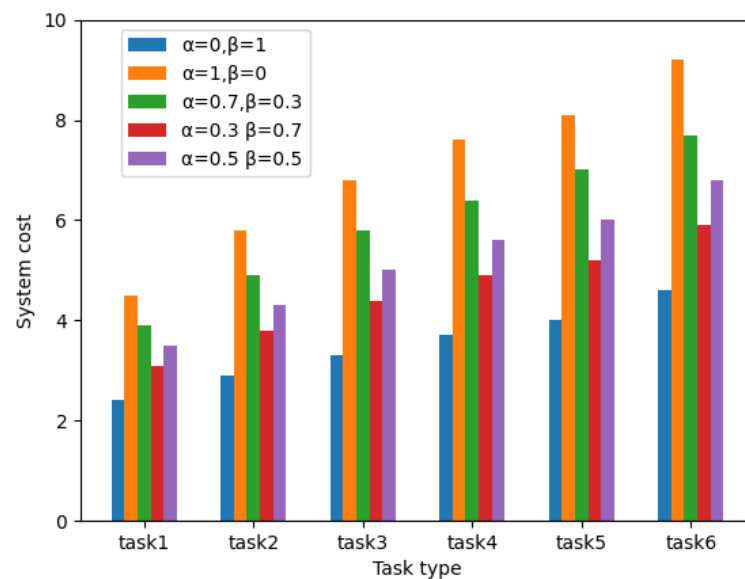


Figure 11. Impact of weight coefficients on cost.

6. Conclusions and Future Work

The strategy investigated in this study transforms the computation offloading problem into a minimal system cost problem, distinguishing itself from prior works. M-GNRL demonstrates the capability to train on graph data to adapt to changing scenarios. In the proposed approach, leveraging graph neural networks enables the agent to make accurate actions in dynamic environments. Moreover, the mobility of devices leads to changes in the graph topology, and corresponding updating algorithms are devised to refresh the graph structure, ultimately determining offloading nodes based on the relational graph. Experimental results indicate the generalization ability of M-GNRL. Compared to various baseline algorithms, M-GNRL effectively reduces both task execution costs and loss rates in MEC scenarios. The simulation experiments are constrained by idealized settings; however, in the future, we aim to test our strategy in real-world scenarios to align with practical usage.

Author Contributions: Conceptualization, T.W., X.O., D.S., Y.C. and H.L.; methodology, T.W. and H.L.; software, T.W.; validation, Y.C., H.L. and D.S.; formal analysis, T.W.; investigation, X.O.; resources, T.W.; data curation, X.O.; writing—original draft preparation, T.W. and D.S.; writing—review and editing, T.W. and H.L.; visualization, Y.C.; supervision, H.L.; project administration, H.L.; funding acquisition, H.L. All authors have read and agreed to the published version of the manuscript.

Funding: This work is supported by the 2023 Opening Research Fund of Yunnan Key Laboratory of Digital Communications (YNJTKFB-20230686, YNKLDC-KFKT-202304) and Yunnan Provincial Major Science and Technology Project: Research and Application of Key Technologies for Scale Processing of Yunnan Characteristic Pre-Prepared Food (Grant No. 202202AE090019).

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Ibn-Khedher, H.; Laroui, M.; Mabrouk, M.B.; Moungla, H.; Afifi, H.; Oleari, A.N.; Kamal, A.E. Edge computing assisted autonomous driving using artificial intelligence. In Proceedings of the 2021 International Wireless Communications and Mobile Computing (IWCMC), Harbin, China, 28 June–2 July 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 254–259.
2. Liu, J.; Zhang, Q. To improve service reliability for AI-powered time-critical services using imperfect transmission in MEC: An experimental study. *IEEE Internet Things J.* **2020**, *7*, 9357–9371. [\[CrossRef\]](#)
3. Mao, Y.; You, C.; Zhang, J.; Huang, K.; Letaief, K.B. A survey on mobile edge computing: The communication perspective. *IEEE Commun. Surv. Tutor.* **2017**, *19*, 2322–2358. [\[CrossRef\]](#)
4. Zhang, J.; Zhao, X. An overview of user-oriented computation offloading in mobile edge computing. In Proceedings of the 2020 IEEE World Congress on Services (SERVICES), Beijing, China, 18–23 October 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 75–76.
5. Noor, T.H.; Zeadally, S.; Alfazi, A.; Sheng, Q.Z. Mobile cloud computing: Challenges and future research directions. *J. Netw. Comput. Appl.* **2018**, *115*, 70–85. [\[CrossRef\]](#)
6. You, C.; Huang, K.; Chae, H.; Kim, B.H. Energy-efficient resource allocation for mobile-edge computation offloading. *IEEE Trans. Wirel. Commun.* **2016**, *16*, 1397–1411. [\[CrossRef\]](#)
7. Wang, Y.; Sheng, M.; Wang, X.; Wang, L.; Li, J. Mobile-edge computing: Partial computation offloading using dynamic voltage scaling. *IEEE Trans. Commun.* **2016**, *64*, 4268–4282. [\[CrossRef\]](#)
8. Bi, S.; Zhang, Y.J. Computation rate maximization for wireless powered mobile-edge computing with binary computation offloading. *IEEE Trans. Wirel. Commun.* **2018**, *17*, 4177–4190. [\[CrossRef\]](#)
9. Wang, F.; Xu, J.; Wang, X.; Cui, S. Joint offloading and computing optimization in wireless powered mobile-edge computing systems. *IEEE Trans. Wirel. Commun.* **2017**, *17*, 1784–1797. [\[CrossRef\]](#)
10. Zhang, W.; Wen, Y.; Guan, K.; Kilper, D.; Luo, H.; Wu, D.O. Energy-optimal mobile cloud computing under stochastic wireless channel. *IEEE Trans. Wirel. Commun.* **2013**, *12*, 4569–4581. [\[CrossRef\]](#)
11. Chen, M.H.; Liang, B.; Dong, M. Joint offloading decision and resource allocation for multi-user multi-task mobile cloud. In Proceedings of the 2016 IEEE International Conference on Communications (ICC), Kuala Lumpur, Malaysia, 22–27 May 2016; IEEE: Piscataway, NJ, USA, 2016; pp. 1–6.
12. Dinh, T.Q.; Tang, J.; La, Q.D.; Quek, T.Q. Offloading in mobile edge computing: Task allocation and computational frequency scaling. *IEEE Trans. Commun.* **2017**, *65*, 3571–3584.
13. Zhan, Y.; Guo, S.; Li, P.; Zhang, J. A deep reinforcement learning based offloading game in edge computing. *IEEE Trans. Comput.* **2020**, *69*, 883–893. [\[CrossRef\]](#)
14. Wu, Z.; Pan, S.; Chen, F.; Long, G.; Zhang, C.; Philip, S.Y. A comprehensive survey on graph neural networks. *IEEE Trans. Neural Netw. Learn. Syst.* **2020**, *32*, 4–24. [\[CrossRef\]](#)
15. Yick, J.; Mukherjee, B.; Ghosal, D. Wireless sensor network survey. *Comput. Netw.* **2008**, *52*, 2292–2330. [\[CrossRef\]](#)
16. Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; Yin, D. Graph neural networks for social recommendation. In Proceedings of the World Wide Web Conference, San Francisco, CA, USA, 13–17 May 2019; pp. 417–426.
17. Glorionnec, P.Y. Reinforcement learning: An overview. In Proceedings of the European Symposium on Intelligent Techniques (ESIT-00), Aachen, Germany, 14–15 September 2000; pp. 14–15.
18. Ding, S.; Lin, D.; Zhou, X. Graph convolutional reinforcement learning for dependent task allocation in edge computing. In Proceedings of the 2021 IEEE International Conference on Agents (ICA), Kyoto, Japan, 13–15 December 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 25–30.
19. Li, Y.; Jiang, C. Distributed task offloading strategy to low load base stations in mobile edge computing environment. *Comput. Commun.* **2020**, *164*, 240–248. [\[CrossRef\]](#)
20. Gao, Y.; Li, Z. Load balancing aware task offloading in mobile edge computing. In Proceedings of the 2022 IEEE 25th International Conference on Computer Supported Cooperative Work in Design (CSCWD), Hangzhou, China, 4–6 May 2022; IEEE: Piscataway, NJ, USA, 2022; pp. 1209–1214.

21. Lu, H.; Gu, C.; Luo, F.; Ding, W.; Liu, X. Optimization of lightweight task offloading strategy for mobile edge computing based on deep reinforcement learning. *Future Gener. Comput. Syst.* **2020**, *102*, 847–861. [\[CrossRef\]](#)
22. Li, J.; Gao, H.; Lv, T.; Lu, Y. Deep reinforcement learning based computation offloading and resource allocation for MEC. In Proceedings of the 2018 IEEE Wireless Communications and Networking Conference (WCNC), Barcelona, Spain, 15–18 April 2018; IEEE: Piscataway, NJ, USA, 2018; pp. 1–6.
23. Liu, Y.; Xie, S.; Zhang, Y. Cooperative offloading and resource management for UAV-enabled mobile edge computing in power IoT system. *IEEE Trans. Veh. Technol.* **2020**, *69*, 12229–12239. [\[CrossRef\]](#)
24. Yan, J.; Bi, S.; Zhang, Y.J.A. Offloading and resource allocation with general task graph in mobile edge computing: A deep reinforcement learning approach. *IEEE Trans. Wirel. Commun.* **2020**, *19*, 5404–5419. [\[CrossRef\]](#)
25. Wang, J.; Hu, J.; Min, G.; Zhan, W.; Ni, Q.; Georgalas, N. Computation offloading in multi-access edge computing using a deep sequential model based on reinforcement learning. *IEEE Commun. Mag.* **2019**, *57*, 64–69. [\[CrossRef\]](#)
26. Zou, J.; Hao, T.; Yu, C.; Jin, H. A3C-DO: A regional resource scheduling framework based on deep reinforcement learning in edge scenario. *IEEE Trans. Comput.* **2020**, *70*, 228–239. [\[CrossRef\]](#)
27. Wang, J.; Hu, J.; Min, G.; Zhan, W.; Zomaya, A.Y.; Georgalas, N. Dependent task offloading for edge computing based on deep reinforcement learning. *IEEE Trans. Comput.* **2021**, *71*, 2449–2461. [\[CrossRef\]](#)
28. Li, C.; Xia, J.; Liu, F.; Li, D.; Fan, L.; Karagiannidis, G.K.; Nallanathan, A. Dynamic offloading for multiuser multi-CAP MEC networks: A deep reinforcement learning approach. *IEEE Trans. Veh. Technol.* **2021**, *70*, 2922–2927. [\[CrossRef\]](#)
29. Tang, M.; Wong, V.W. Deep reinforcement learning for task offloading in mobile edge computing systems. *IEEE Trans. Mob. Comput.* **2020**, *21*, 1985–1997. [\[CrossRef\]](#)
30. Chen, T.; Zhang, X.; You, M.; Zheng, G.; Lambotaran, S. A GNN-based supervised learning framework for resource allocation in wireless IoT networks. *IEEE Internet Things J.* **2021**, *9*, 1712–1724. [\[CrossRef\]](#)
31. He, S.; Xiong, S.; Ou, Y.; Zhang, J.; Wang, J.; Huang, Y.; Zhang, Y. An overview on the application of graph neural networks in wireless networks. *IEEE Open J. Commun. Soc.* **2021**, *2*, 2547–2565. [\[CrossRef\]](#)
32. Li, K.; Ni, W.; Yuan, X.; Noor, A.; Jamalipour, A. Deep-graph-based reinforcement learning for joint cruise control and task offloading for aerial edge internet of things (edgeiot). *IEEE Internet Things J.* **2022**, *9*, 21676–21686. [\[CrossRef\]](#)
33. Sun, Z.; Mo, Y.; Yu, C. Graph-reinforcement-learning-based task offloading for multiaccess edge computing. *IEEE Internet Things J.* **2021**, *10*, 3138–3150. [\[CrossRef\]](#)
34. Nayak, S.; Choi, K.; Ding, W.; Dolan, S.; Gopalakrishnan, K.; Balakrishnan, H. Scalable multi-agent reinforcement learning through intelligent information aggregation. In Proceedings of the International Conference on Machine Learning, Honolulu, HI, USA, 23–29 July 2023; pp. 25817–25833.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.